Detection of Deep Learning Facial Forgery Using Convolutional Neural Networks

Jesse A. Mendoza

Abstract:

There is an increasing blurring of what is real and what is artificially generated when it comes to online media. With the development of advanced Deep Learning techniques, images can be manipulated so that the person's facial features are replaced with those of another. The results that can be achieved are often quite impressive and possess with a concerning level of realism [1]. As such, there is growing interest in detecting this form of synthetic media to avoid its application in facial forgery. My aim is two-fold; create an algorithm for facial detection and create a neural network that can discriminate between authentic faces and artificially generated faces. First, I implemented the Haar Cascade/ Viola-Jones algorithm using OpenCV to extract faces within an input image. Subsequently, I used Tensorflow's Keras package to construct a convolutional neural network. I trained the network with 38,570 deepfake faces and 38,571 real faces that span over four different datasets [2,3,4,5]. When the model was evaluated on a separate set of 5,403 deepfake faces and 5,401 real faces, the model was able to achieve an overall binary classification accuracy of 76%. Additionally, while the model was able to detect 97% of cases where the image was a deepfake, only 68% of its predictions of a face being a deepfake were accurate. Conversely, the model was only able to detect 55% of the cases where the image was of a real face but had a 95% accuracy in its prediction of a face being a real face. Therefore, the model's predictions had a high rate of false positives when it came to deepfakes but a low rate when it came to real faces, and the model had a high rate of false negatives when it came to real faces but a low rate of false negatives when it came to deepfakes.

Introduction:

Whether it be misinformation or simply isolation, or both, there is a growing incongruence in terms of what information people deem to be real. This extends from the possibility of deliberately fictious texts to subtly altered images and videos. With the paradigm shift towards the use of neural networks, deepfakes have become a growing concern in its potential to make realistic transformations from one face to another. I was thus interested in both understanding how well current detection algorithms catch deepfakes and the basics of algorithms using neural networks. As I have never designed nor implemented a neural network in the past, my first key challenge was learning the fundamentals of convolutional neural networks and the use of TensorFlow's Keras API in implementing it. While much research was needed to get a good understanding of the content, the biggest challenge ended up being the gathering of various datasets to provide enough data for the neural network. The majority of deepfake content online is in video format, and my concern was that having a huge collection of frames of the same datapoint could hamper learning. Thus, I set out to gather images of deepfakes and real faces that span a large pool of heterogenous faces. The resulting trained model seemed to be in alignment with previous reports [6].

Background:

Deepfake utilizes deep learning via neural networks to transform an individual's face to that of another. The generation of this form of synthetic media became more publicly accessible with emergence of applications like FakeApp [7]. As this technique became more refined, the algorithm Face2Face introduced the capability of transferring expressions of one face to another in real-time [8]. This continual advancement in these techniques has simultaneously increased the concerns of its lawful usage.

From 2019 to 2020 various big tech companies such as Facebook, Microsoft, AWS, and the Partnership on AI's Media Integrity Steering Committee collectively created the DeepFake Detection Challenge on Kaggle, wherein the team that produces the best deepfake detection model would win a prize of 1M$ [9]. This led to the creation of the largest video dataset of deepfakes from a range of participants. Amongst the models submitted the leading model obtained only a 65% accuracy on the 4000 videos that were used as a holdout set for testing [10]. This model implemented multitask cascaded convolutional neural networks that extracted sample faces from static frames and encoded its features. This suggested that the usage of neural networks could be the right direction in designing an algorithm to detect deepfake faces.

Afchar et al., designed a deep neural network that focused on a mesoscopic level of analysis of an image and was constructed using only several convolutional layers followed by a dense hidden layer. The group called this architechture Meso-4 and was able to achieve a 93.2% detection rate of deepfake faces when using intra-frames of videos within their dataset [6]. I thus set out to create a convolutional neural network that would use the architecture of Meso-4 and train this algorithm on a variety of real and deepfake faces with the end goal being a pipeline through which an image would be used as input and the resulting output would be every extracted face in the image with a label as to whether the face is a deep fake or not. Essentially, my goal is to recreate and analyze the performance of theMeso-4 architecture.

Methodology:

Facial detection.

To detect and extract faces within an image I utilized a Viola Jones detection algorithm with a Haar-Cascade. This algorithm makes use Haar-like features, which are features obtained by the difference of the summed value across various subregions in a kernel. These features are then used by AdaBoost, which is a learning algorithm that makes use of multiple predictors to correct prior misclassifications. Finally, this runs within a cascade filter, such that the image passes through a series of binary predictors of each of the learned features. Since this is within a series, a single classifier determining that a feature is not present in the image is sufficient to classify the image as not a face.

As my focus within this project is the creation of the neural network, I implemented the Viola Jones algorithm by utilizing the haarcascade_frontalface_alt.xml model provided in OpenCV that has been pretrained on a variety of frontal facing faces. This is then utilized within a Cascade classifier to accomplish the Haar-Cascade process described above. The final face detection algorithm used takes an image of any size, detects multiple faces within the image, and then resizes that image to 150x150 to conform to the neural networks input dimension.

Datasets.

In total the dataset consists of 87,945 images, of which 77,141 images were used to train the neural network and 10,804 images were set aside for evaluating the model's performance. Within the training dataset, 38,570 of the images were of deepfake faces and 38,571 of the images were of real faces. Within the testing dataset, 5,403 of the images were of deepfake faces and 5,401 of the images were of real faces. All images that were not already 150x150 were resized to match the input parameters of the neural network. The images were placed in folders labeled test or train with subfolders of either deep_faces or real_faces for deepfake faces and real faces respectively. Furthermore, a subset of the images that came from one of the databases were not exclusively of an individual face. Therefore, some of the images I used the facial detection algorithm previously described to extract relevant faces. I then manually checked the outputs to remove false-positives, i.e not a face. All the images come from 4 different datasets [2,3,4,5]. The deepfake faces come from both the FaceForensic dataset and the frames derived from the DeepFake Detection Challenge dataset. The real faces come from the LFW, UTK, and DeepFake Detection Challenge datasets.

Neural Network Architecture.

The convolutional neural network was constructed using the Keras API of TensorFlow. The neural network is composed of an input layer, 4 convolutional layers, a densely connected hidden layer, and finally an output layer (*Figure 1*.). The input layer has dimensions 150x150x3 to accept colored images that are 150x15. The convolutional layers all utilize ReLu activation functions as a means of converting the inputs to a node to an output. The first convolutional layer is composed of 8 nodes with 3x3 kernels, the second layer is composed of 8 nodes with 5x5 kernels, the third and fourth layer is composed of 16 nodes with 5x5 kernels. Between each of the convolutional layers are pairs of Batch normalization and 2x2 max pooling layers (this takes the maximum value within a 2x2 kernel), except for the fourth layer which is followed by a batch normalization layer and a 4x4 max pooling layer. Next the inputs are each flattened into singular dimensions and fed into a densely connected layer of 16 nodes with a randomized 0.5 dropout on the inputs to reduce changes to overfit the training data. After which the inputs go through another 0.5 dropout and then into a final node that takes the inputs, uses a sigmoid function to output a value between 1 and 0, wherein 1 implies a real face and 0 implies a deepfake face. The total number of parameters defined within the model is 15,785.

Training.

The images within the dataset were fed into the model via Kera's image data generators, which allows for the usage of preprocessing of images and data augmentation within a pipeline. Both the training and testing generators normalize the images. The generator for the training data was made to use various forms of data augmentation to help with training generalizability (width shift, height shift, shearing, brightness shift) and set aside 20% of the generated images for validation purposes between epochs of training. Both the training and validation generators shuffle the inputs within a batch, however the testing generator does not as to avoid a mismatch with the order of labels. The model was compiled to use a binary cross-entropy loss function to minimize using the Adam optimizer provided in Keras. The learning rate was set to 1e-3. The model was trained to a max of 30 epochs with batch size of 32. I additionally allowed for an early stopping function, such that if the accuracy ceases to improve

after 5 epochs, the training will stop since we want to avoid overfitting the data. Training finished after 13 epochs, such that the model at epoch 8 is the ideal set of parameters.

Results:

With the convolutional neural network trained, the final algorithm functions by taking an input image, processing the image to detect faces, extract the faces, feed it into the model, and output a number between 0 and 1, a sample example of the algorithm process is shown in Figure 2. The classification is such that values above or equal to 0.5 are labeled as unaltered and values below 0.5 are labelled as deepfakes.

The model was additionally evaluated by using an Image Generator that created a pipeline to feed the test samples into the model. The results of the evaluation can be seen in Figure 3. the model was able to achieve an overall binary classification accuracy of 76% (Figure 3C). While the model was able to detect 97% of cases where the image was a deepfake (recall), only 68% of its predictions of a face being a deepfake were accurate (precision). Conversely, the model was only able to detect 55% of the cases where the image was of a real face (recall) but had a 95% accuracy in its prediction of a face being a real face (precision). As such the model's predictions had a high rate of false positives when it came to deepfakes but a low rate when it came to real faces, and the model had a high rate of false negatives when it came to real faces but a low rate of false negatives when it came to deepfakes. The F-1 simply represents the harmonic mean between recall and precision values, wherein the score was 0.80 for deepfake face classifications and 0.69 for real face classifications. These results are further exemplified in the confusion matrix (Figure 3A), wherein the true positive values (top-left, bottom-right) are darker (i.e more samples properly classified) than the opposite diagonal, however the asymmetry in the opposite diagonal shows that deepfake faces were rarely categorized as real faces (top-right), however it was more frequent that real faces were categorized as deepfake faces (bottom-left). Finally, the ROC curve was derived for the predictions made by the model on the test images (Figure 3 B). The AUC for the ROC is 0.947, implying that the model can effectively discriminate between the distribution of values representing deepfake faces and the distribution of values representing real faces.

Discussion:

The final overall accuracy of the model was 76%, which is quite high given this classification problem. However, it seems like the model is quite liberal with its classification of an image as a deepfake given by the fact so many images were classified as such incorrectly and many of the real faces were missed by the model. Afchar et al., had stated in their paper that they obtained a detection rate of 93.2% for the deepfake images using Meso-4, and unfortunately never report the accuracy, precision, nor recall values of their models explicitly. Assuming that the detection rate being implied in their work is corresponding to the recall rate of deepfake images, then my model is roughly in accordance with their results (97% recall on deepfake faces). One major limitation of my work one of the deepfake face sets contains multiples of the same face but in different positions. While still useful, having a more heterogenous dataset would help with generalizability. To extend this project it would be interesting to have the algorithm be able to make classifications in real-time via video input.

Layer (type)	Output Shape	Param #
image (InputLayer)	[(None, 150, 150, 3)]	0
conv2d (Conv2D)	(None, 150, 150, 8)	224
batch_normalization (BatchNo	(None, 150, 150, 8)	32
max_pooling2d (MaxPooling2D)	(None, 75, 75, 8)	0
conv2d_1 (Conv2D)	(None, 75, 75, 8)	1608
batch_normalization_1 (Batch	(None, 75, 75, 8)	32
max_pooling2d_1 (MaxPooling2	(None, 37, 37, 8)	0
conv2d_2 (Conv2D)	(None, 37, 37, 16)	3216
batch_normalization_2 (Batch	(None, 37, 37, 16)	64
max_pooling2d_2 (MaxPooling2	(None, 18, 18, 16)	0
conv2d_3 (Conv2D)	(None, 18, 18, 16)	6416
batch_normalization_3 (Batch	(None, 18, 18, 16)	64
max_pooling2d_3 (MaxPooling2	(None, 4, 4, 16)	0
flatten (Flatten)	(None, 256)	0
dropout (Dropout)	(None, 256)	0
dense (Dense)	(None, 16)	4112
dropout_1 (Dropout)	(None, 16)	0
model_output (Dense)	(None, 1)	17

Figure 1. Architecture of the neural network.

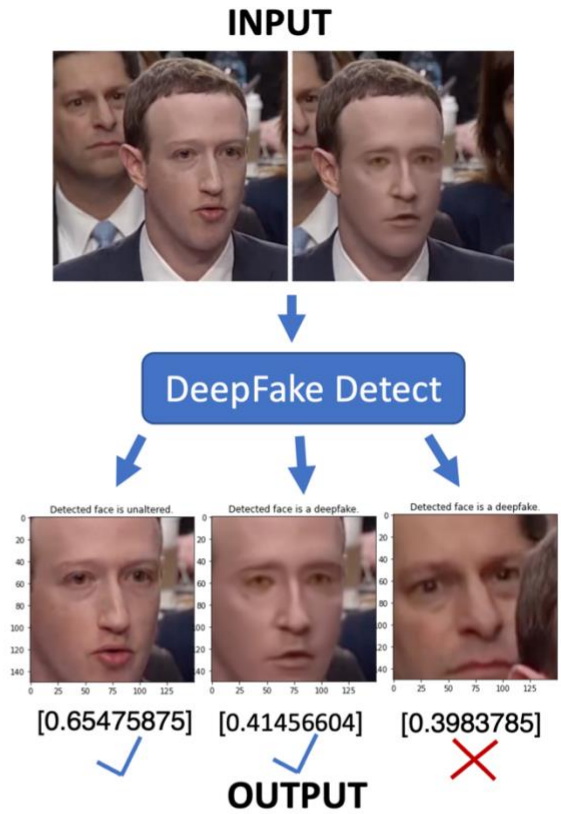
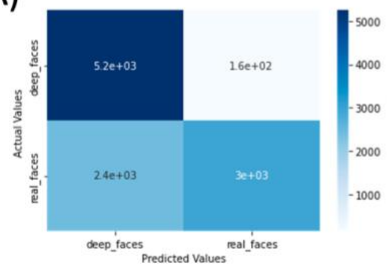
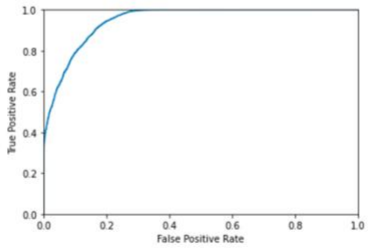


Figure 2. Sample out of a sample input image
Left and center are correct.
Right is a false-positive

A)



B)



C)

Classification Report

	precision	recall	f1-score	support
deep_faces	0.68	0.97	0.80	5402
real_faces	0.95	0.55	0.69	5400
accuracy			0.76	10802
macro avg	0.82	0.76	0.75	10802
weighted avg	0.82	0.76	0.75	10802

Figure 3. Evaluation results (A). Confusion matrix for predictions (B). ROC curve based on predictions (C). Classification measures

References

[1] Robert Chesney and Danielle Keats Citron, **"Deep Fakes: A Looming Challenge for Privacy, Democracy, and National Security"** 107 California Law Review (2019, Forthcoming); U of Texas Law, Public Law Research Paper No. 692; U of Maryland Legal Studies Research Paper No. 2018-21.

[2] FaceForensics Dataset https://www.kaggle.com/datasets/greatgamedota/faceforensics

[3] LFW Dataset http://vis-www.cs.umass.edu/lfw/

[4] UTK Face Dataset https://susanqq.github.io/UTKFace/

[5] DFDC Frames database https://www.kaggle.com/datasets/dagnelies/deepfake-faces

[6] Darius Afchar, Vincent Nozick, Junichi Yamagishi, and Isao Echizen, **"Mesonet: a compact facial video forgery detection network,"** in IEEE International Workshop on Information Forensics and Security (WIFS), 2018.

[7] DeepSwap (FakeApp) https://blog.deepswap.ai/

[8] Face2Face https://github.com/datitran/face2face-demo

[9] DFDC Kaggle https://www.kaggle.com/c/deepfake-detection-challenge

[10] Matthew Groh, Ziv Epstein, Chaz Firestone, and Rosalind Picard **"Deepfake detection by human crowds, machines, and machine-informed crowds"**, PNAS, 2021.